

PROJECT-1: 2-TIER WEB APPLICATION INFRASTRUCTURE ON AWS

Objective: The objective of this project is to automate the deployment of AWS infrastructure using Terraform. The infrastructure includes two public web server and a private database server inside a custom VPC. Terraform creates all required AWS resources with a single terraform apply command.

Tools and Services used

- Terraform
- AWS EC2
- AWS VPC
- AWS S3 (Remote Backend)
- Internet Gateway
- Route Tables
- Security Groups
- Amazon Linux 2

Project Implementation

Step 1: Created Terraform Files

Created the following Files:

Provider.tf, backend.tf, main.tf, variable.tf, terraform.tfvars, outputs.tf, userdata.sh, README.md

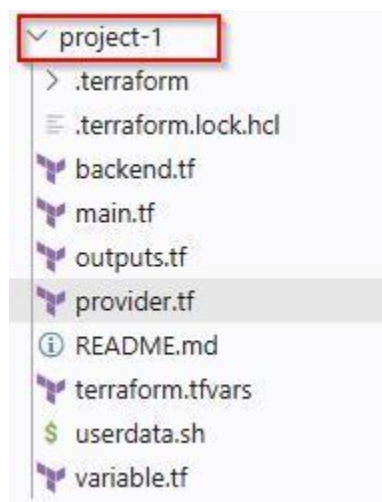


Figure 1: Terraform project directory structure.

Step 2: Provider Configuration



Figure 2: AWS provider configuration.

Step 3: Backend Configuration

```
project-1 > backend.tf
1 terraform {
2   backend "s3" {
3     bucket = "terraform-state-bucket1-project1"
4     key    = "terraform-project-1/terraform.tfstate"
5     region = "ap-south-1"
6     use_lockfile = true
7   }
8 }
9
```

Figure 3: Remote backend configuration using Amazon S3

Step 4: Variables

```
project-1 > variable.tf
1 variable "aws_region" {
2   description = "aws region for resources"
3   type        = string
4   default     = "ap-south-1"
5 }
6
7 variable "vpc_cidr" {
8   description = "cidr block for vpc"
9   type        = string
10  default     = "10.0.0.0/16"
11 }
12
13 variable "public_subnet" {
14   description = "cidr block for public subnet"
15   type        = string
16   default     = "10.0.1.0/24"
17 }
18
19 variable "private_subnet" {
20   description = "cidr block for private subnet"
21   type        = string
22   default     = "10.0.2.0/24"
23 }
24
25 variable "availability_zone" {
26   description = "availability zone"
27   type        = string
28   default     = "ap-south-1a"
29 }
30
31 variable "instance_type" {
32   description = "EC2 instance type"
33   type        = string
34   default     = "t3.micro"
35 }
36
37 variable "my_ip" {
38   description = "my public IP"
39   type        = string
40 }
41
42 variable "owner" {
43   description = "resource owner"
44   type        = string
45   default     = "Pavithra Machha"
46 }
47
48 variable "project_name" {
49   description = "project name"
50   type        = string
51   default     = "terraform-project-1"
52 }
53
54 variable "Environment" {
55   description = "environment name"
56   type        = string
57   default     = "dev"
58 }
59
60 variable "key_name" {
61   description = "key pair name"
62   type        = string
63   default     = "web-server-key"
64 }
65
66 variable "public_subnet_2" {
67   description = "CIDR block for second public subnet"
68   type        = string
69   default     = "10.0.3.0/24"
70 }
71
72 variable "availability_zone_2" {
73   description = "Availability Zone for second public subnet"
74   type        = string
75   default     = "ap-south-1b"
76 }
77
78 }
```

Figure 4: Terraform variable declarations.

Description:

The variables.tf file stores the input values required for the Terraform project. It helps keep the code organized and allows values to be changed easily without editing the main Terraform code.

Step 5: Variable Values

```
project-1 > terraform.tfvars
1 aws_region      = "ap-south-1"
2 vpc_cidr        = "10.0.0.0/16"
3 public_subnet   = "10.0.1.0/24"
4 private_subnet  = "10.0.2.0/24"
5 availability_zone = "ap-south-1a"
6 instance_type   = "t3.micro"
7 my_ip           = "203.0.113.25/32"
8 owner           = "Pavithra Machha"
9 project_name    = "terraform-project-1"
10 Environment     = "dev"
11 key_name        = "web-server-key"
12 public_subnet_2 = "10.0.3.0/24"
13 availability_zone_2 = "ap-south-1b"
```

Figure 5: Variable values used for deployment.

Step 6: Main Terraform Code

VPC

```
project-1 > main.tf
1 resource "aws_vpc" "main" {
2   cidr_block = var.vpc_cidr
3   tags = {
4     Name       = "${var.project_name}-vpc"
5     Environment = var.Environment
6     Owner      = var.owner
7   }
8 }
```

Figure 6: Terraform code for VPC creation.

Public & Private Subnets

```
10 resource "aws_subnet" "public_subnet" {
11   vpc_id            = aws_vpc.main.id
12   cidr_block        = var.public_subnet
13   availability_zone  = var.availability_zone
14   map_public_ip_on_launch = true
15   tags = {
16     Name       = "${var.project_name}-public_subnet"
17     Environment = var.Environment
18     Owner      = var.owner
19   }
20 }
21
22 resource "aws_subnet" "public_subnet_2" {
23   vpc_id            = aws_vpc.main.id
24   cidr_block        = var.public_subnet_2
25   availability_zone  = var.availability_zone_2
26   map_public_ip_on_launch = true
27   tags = {
28     Name       = "${var.project_name}-public_subnet_2"
29     Environment = var.Environment
30     Owner      = var.owner
31   }
32 }
33
34 resource "aws_subnet" "private_subnet" {
35   vpc_id            = aws_vpc.main.id
36   cidr_block        = var.private_subnet
37   availability_zone  = var.availability_zone
38   tags = {
39     Name       = "${var.project_name}-private_subnet"
40     Environment = var.Environment
41     Owner      = var.owner
42   }
43 }
44
45 }
```

Figure 7: Terraform code for subnet creation.

Internet Gateway , Route Table&and it's association

```
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags = {
    Name       = "${var.project_name}-igw"
    Environment = var.Environment
    Owner      = var.owner
  }
}

resource "aws_route_table" "public_subnet" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = {
    Name       = "${var.project_name}-public-rt"
    Environment = var.Environment
    Owner      = var.owner
  }
}

resource "aws_route_table_association" "public_subnet" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_subnet.id
}

resource "aws_route_table_association" "public_subnet_2" {
  subnet_id      = aws_subnet.public_subnet_2.id
  route_table_id = aws_route_table.public_subnet.id
}
```

Figure 8: Terraform code for Internet Gateway , routing& association.

Security Groups

```
61 resource "aws_security_group" "web_sg" {
62   name_prefix = "${var.project_name}-web_sg"
63   vpc_id      = aws_vpc.main.id
64
65   ingress {
66     from_port = 22
67     to_port   = 22
68     protocol  = "tcp"
69     cidr_blocks = [var.my_ip]
70   }
71
72   ingress {
73     from_port = 80
74     to_port   = 80
75     protocol  = "tcp"
76     cidr_blocks = ["0.0.0.0/0"]
77   }
78
79   egress {
80     from_port = 0
81     to_port   = 0
82     protocol  = "-1"
83     cidr_blocks = ["0.0.0.0/0"]
84   }
85   tags = {
86     Name      = "${var.project_name}-web_sg"
87     Environment = var.Environment
88     Owner      = var.owner
89   }
90 }
91
46 resource "aws_security_group" "db_sg" {
47   name_prefix = "${var.project_name}-db_sg"
48   vpc_id      = aws_vpc.main.id
49
50   ingress {
51     from_port = 3306
52     to_port   = 3306
53     protocol  = "tcp"
54     security_groups = [aws_security_group.web_sg.id]
55   }
56
57   egress {
58     from_port = 0
59     to_port   = 0
60     protocol  = "-1"
61     cidr_blocks = ["0.0.0.0/0"]
62   }
63   tags = {
64     Name      = "${var.project_name}-db_sg"
65     Environment = var.Environment
66     Owner      = var.owner
67   }
68 }
```

Figure 9: Terraform code for Web and Database Security Groups.

Allocate and Attach an Elastic IP to the Web Server

```
resource "aws_eip" "web_eip" {
  domain = "vpc"

  tags = {
    Name      = "${var.project_name}-web-eip"
    Environment = var.Environment
    Owner      = var.owner
  }
}

resource "aws_eip_association" "web_eip_association" {
  instance_id = aws_instance.web_server.id
  allocation_id = aws_eip.web_eip.id
}
```

Figure 10: Elastic IP Configuration

Description:

Terraform is used to allocate an Elastic IP address and attach it to the web server EC2 instance. This provides a fixed public IP address for accessing the web application.

EC2 Key Pair Configuration

```
resource "aws_key_pair" "web_key" {
  key_name   = var.key_name
  public_key = file("${path.module}/web-server-key.pub")

  tags = {
    Name      = "${var.project_name}-keypair"
    Environment = var.Environment
    Owner      = var.owner
  }
}
```

Figure 11: EC2 Key Pair Configuration

Description:

A key pair was created and attached to the web server EC2 instance. The private key allows secure SSH access to the server without using passwords.

EC2 Instances

```
162 data "aws_ami" "amazon_linux" {
163   most_recent = true
164   owners      = ["amazon"]
165   filter {
166     name   = "name"
167     values = ["amzn2-ami-hvm-*x86_64-gp2"]
168   }
169 }
170
171 resource "aws_instance" "web_server" {
172   ami           = data.aws_ami.amazon_linux.id
173   instance_type = var.instance_type
174   subnet_id     = aws_subnet.public_subnet.id
175   security_groups = [aws_security_group.web_sg.id]
176   key_name      = var.key_name
177   associate_public_ip_address = true
178   user_data     = file("userdata.sh")
179   tags = {
180     Name        = "${var.project_name}-web_server"
181     Environment = var.Environment
182     Owner       = var.owner
183   }
184 }
185
186 resource "aws_instance" "db_server" {
187   ami           = data.aws_ami.amazon_linux.id
188   instance_type = var.instance_type
189   subnet_id     = aws_subnet.private_subnet.id
190   security_groups = [aws_security_group.db_sg.id]
191   key_name      = var.key_name
192   associate_public_ip_address = false
193   tags = {
194     Name        = "${var.project_name}-db_server"
195     Environment = var.Environment
196     Owner       = var.owner
197   }
198 }
199
200 resource "aws_instance" "web_server_2" {
201   ami           = data.aws_ami.amazon_linux.id
202   instance_type = var.instance_type
203   subnet_id     = aws_subnet.public_subnet_2.id
204   security_groups = [aws_security_group.web_sg.id]
205   key_name      = var.key_name
206   associate_public_ip_address = true
207   user_data     = file("userdata.sh")
208   tags = {
209     Name        = "${var.project_name}-web_server_2"
210     Environment = var.Environment
211     Owner       = var.owner
212   }
213 }
214
215 }
```

Figure 12: Terraform code for launching EC2 instances.

Step 7: User Data Script

```
project-1 > $ cat userdata.sh
1  #!/bin/bash
2
3  yum update -y
4
5  yum install -y httpd
6
7  systemctl enable --now httpd
8
9  cat <<EOF > /var/www/html/index.html
10 <!DOCTYPE html>
11 <html>
12 <head>
13   <title>2-TIER WEB APPLICATION INFRASTRUCTURE ON AWS</title>
14 </head>
15
16 <body>
17
18   <h1>Hello from Pavithra Machha web server</h1>
19
20   <p>
21     This web server was provisioned automatically using Terraform.
22   </p>
23
24   <p>
25     Running on AWS EC2 inside a public subnet.
26   </p>
27
28 </body>
29 </html>
30 EOF
```

Figure 13: User data script for web server configuration.

Description:

The userdata.sh script automatically installs the Apache web server, starts the Apache service, and creates a custom index.html page. This script runs automatically when the EC2 instance is launched, so no manual server configuration is required.

Step 8: Outputs

```
project-1 > outputs.tf
1  output "vpc_id" {
2    description = "id of vpc"
3    value       = aws_vpc.main.id
4  }
5
6  output "web_public_ip" {
7    description = "public ip of ec2 instance"
8    value       = aws_eip.web_server.public_ip
9  }
10
11 output "web_public_url" {
12   description = "web server public url"
13   value       = "http://${aws_eip.web_server.public_ip}"
14 }
15
16 output "db_private_ip" {
17   description = "private ip of ec2 instance"
18   value       = aws_instance.db_server.private_ip
19 }
```

Figure 14: Terraform output definition

Deployment

Step 9: Terraform init

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
$ terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.55.0

Terraform has been successfully initialized!
```

Figure 15: Terraform initialization completed successfully.

Step 10: Terraform format

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
$ terraform fmt
terraform.tfvars
```

Figure 16: Terraform format configuration.

Step 11: Terraform validate

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
$ terraform validate
Success! The configuration is valid.
```

Figure 17: Terraform configuration validation.

Step 12: Terraform Plan

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
$ terraform plan
+ "Environment" = "dev"
+ "Name"        = "terraform-project-1-vpc"
+ "Owner"       = "Pavithra Machha"
}
}

Plan: 16 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ db_private_ip = (known after apply)
+ vpc_id        = (known after apply)
+ web_public_ip = (known after apply)
+ web_public_url = (known after apply)
```

Figure 18: Terraform execution plan.

Step 13: Terraform Apply

```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
$ terraform apply
aws_instance.web_server_2: Creation complete after 13s [id=i-05b35163c436d7916]
aws_eip_association.web_eip_association: Creation complete after 1s [id=eipassoc-0c01ca46cba1a0074]

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.
```

Figure 17: Successful infrastructure deployment.

AWS Console Verification

Step14:vpc

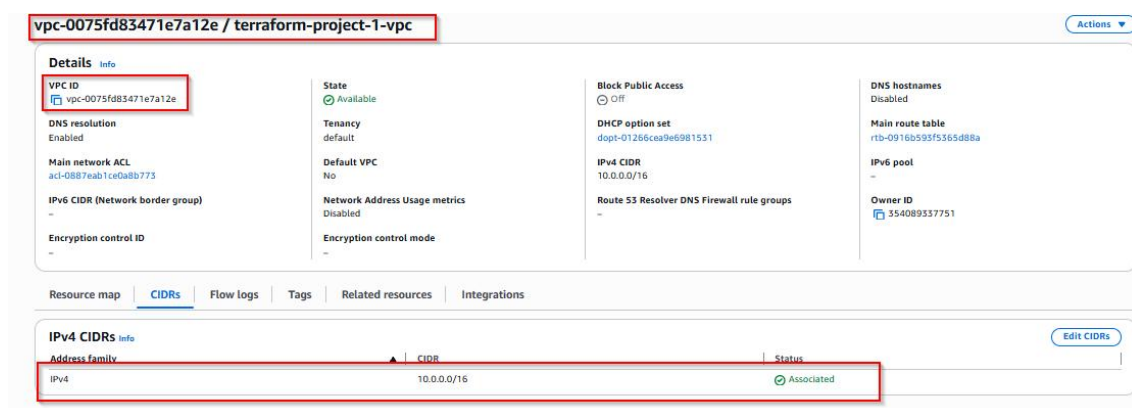
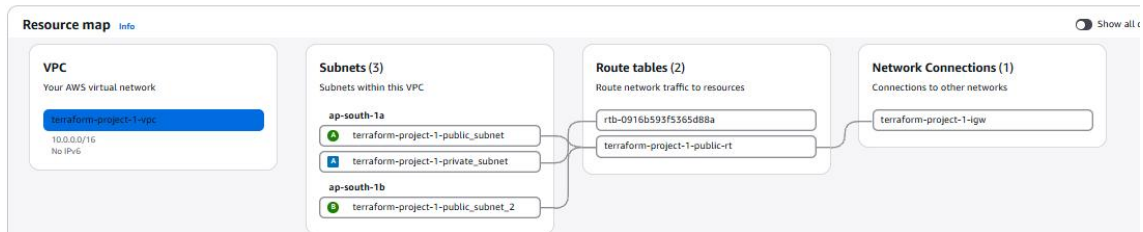


Figure 18: VPC created successfully



Step 15: Subnets

subnet-0c1992819bf044d32 / terraform-project-1-public_subnet

Details

Subnet ID
subnet-0c1992819bf044d32

IPv4 CIDR
10.0.1.0/24

Availability Zone
ap-south-1a

Network ACL
acl-0887eab1c0a8b773

Auto-assign customer-owned IPv4 address
No

IPv6 CIDR reservations
-

Resource name DNS AAAA record
Disabled

Subnet ARN
arn:aws:ec2:ap-south-1:354089337751:subnet/subnet-0c1992819bf044d32

Available IPv4 addresses
250

Network border group
ap-south-1

Default subnet
No

Customer-owned IPv4 pool
-

IPv6-only
No

DNS64
Disabled

State
Available

IPv6 CIDR
-

VPC
vpc-0075fd83471e7a12e | terraform-project-1-vpc

Auto-assign public IPv4 address
Yes

Outpost ID
-

Hostname type
IP name

Owner
354089337751

Block Public Access
Off

IPv6 CIDR association ID
-

Route table
rtb-06fa751832dd16790 | terraform-project-1-public-rt

Auto-assign IPv6 address
No

IPv4 CIDR reservations
-

Resource name DNS A record
Disabled

Flow logs | **Route table** | Network ACL | CIDR reservations | Sharing | Tags | Related resources

Route table: rtb-06fa751832dd16790 / terraform-project-1-public-rt

Routes (2)

Filter routes

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-01e6ae58720f7bb3

subnet-03b29825d7d121330 / terraform-project-1-public_subnet_2

Details

Subnet ID
subnet-03b29825d7d121330

IPv4 CIDR
10.0.3.0/24

Availability Zone
ap-south-1b

Network ACL
acl-0887eab1c0a8b773

Auto-assign customer-owned IPv4 address
No

IPv6 CIDR reservations
-

Resource name DNS AAAA record
Disabled

Subnet ARN
arn:aws:ec2:ap-south-1:354089337751:subnet/subnet-03b29825d7d121330

Available IPv4 addresses
250

Network border group
ap-south-1

Default subnet
No

Customer-owned IPv4 pool
-

IPv6-only
No

DNS64
Disabled

State
Available

IPv6 CIDR
-

VPC
vpc-0075fd83471e7a12e | terraform-project-1-vpc

Auto-assign public IPv4 address
Yes

Outpost ID
-

Hostname type
IP name

Owner
354089337751

Block Public Access
Off

IPv6 CIDR association ID
-

Route table
rtb-06fa751832dd16790 | terraform-project-1-public-rt

Auto-assign IPv6 address
No

IPv4 CIDR reservations
-

Resource name DNS A record
Disabled

Flow logs | **Route table** | Network ACL | CIDR reservations | Sharing | Tags | Related resources

Route table: rtb-06fa751832dd16790 / terraform-project-1-public-rt

Routes (2)

Filter routes

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-01e6ae58720f7bb3

subnet-0a41f362190095ca5 / terraform-project-1-private_subnet Actions

Details
Subnet ID
[subnet-0a41f362190095ca5](#)
IPv4 CIDR
[10.0.2.0/24](#)
Availability Zone
[ap-south-1a](#)
Network ACL
[acl-0887eab1ca3a8b773](#)
Auto-assign customer-owned IPv4 address
No
IPv6 CIDR reservations
-
Resource name DNS AAAA record
Disabled

Subnet ARN
[arn:aws:ec2:ap-south-1:354089337751:subnet/subnet-0a41f362190095ca5](#)
Available IPv4 addresses
250
Network border group
[ap-south-1](#)
Default subnet
No
Customer-owned IPv4 pool
-
IPv6-only
No
DNS64
Disabled

State
Available
IPv6 CIDR
-
VPC
[vpc-0075fd83471e7a12e | terraform-project-1-vpc](#)
Auto-assign public IPv4 address
No
Outpost ID
-
Hostname type
IP name
Owner
[354089337751](#)

Block Public Access
Off
IPv6 CIDR association ID
-
Route table
[rtb-0916b593f5365d88a](#)
Auto-assign IPv6 address
No
IPv4 CIDR reservations
-
Resource name DNS A record
Disabled

Flow logs | **Route table** | Network ACL | CIDR reservations | Sharing | Tags | Related resources

Route table: [rtb-0916b593f5365d88a](#) Edit route table association

Routes (1)

Filter routes

Destination	Target
10.0.0.0/16	local

Figure 19: Public and Private Subnets.

Step 16: Internet Gateway

igw-01e0ae58720ff7bb3 / terraform-project-1-igw Actions

Details
Internet gateway ID
[igw-01e0ae58720ff7bb3](#)
State
Attached
VPC ID
[vpc-0075fd83471e7a12e | terraform-project-1-vpc](#)
Owner
[354089337751](#)

Tags (3)
Search tags

Key	Value
Environment	dev
Owner	Pavithra Machha
Name	terraform-project-1-igw

Figure 20: Internet Gateway attached to the VPC.

Step 17: Route Table

rtb-06fa751832dd16790 / terraform-project-1-public-rt Actions

Details
Route table ID
[rtb-06fa751832dd16790](#)
VPC
[vpc-0075fd83471e7a12e | terraform-project-1-vpc](#)
Main
No
Owner ID
[354089337751](#)
Explicit subnet associations
2 subnets
Edge associations
-

Routes | **Subnet associations** | Edge associations | Route propagation | Tags | Related resources

Explicit subnet associations (2) Edit subnet associations

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
terraform-project-1-public_subnet_2	subnet-03b29825d7d121530	10.0.3.0/24	-
terraform-project-1-public_subnet	subnet-0c1992819bf04d432	10.0.1.0/24	-

Figure 21: Public Route Table configuration.

Step18: Security Groups

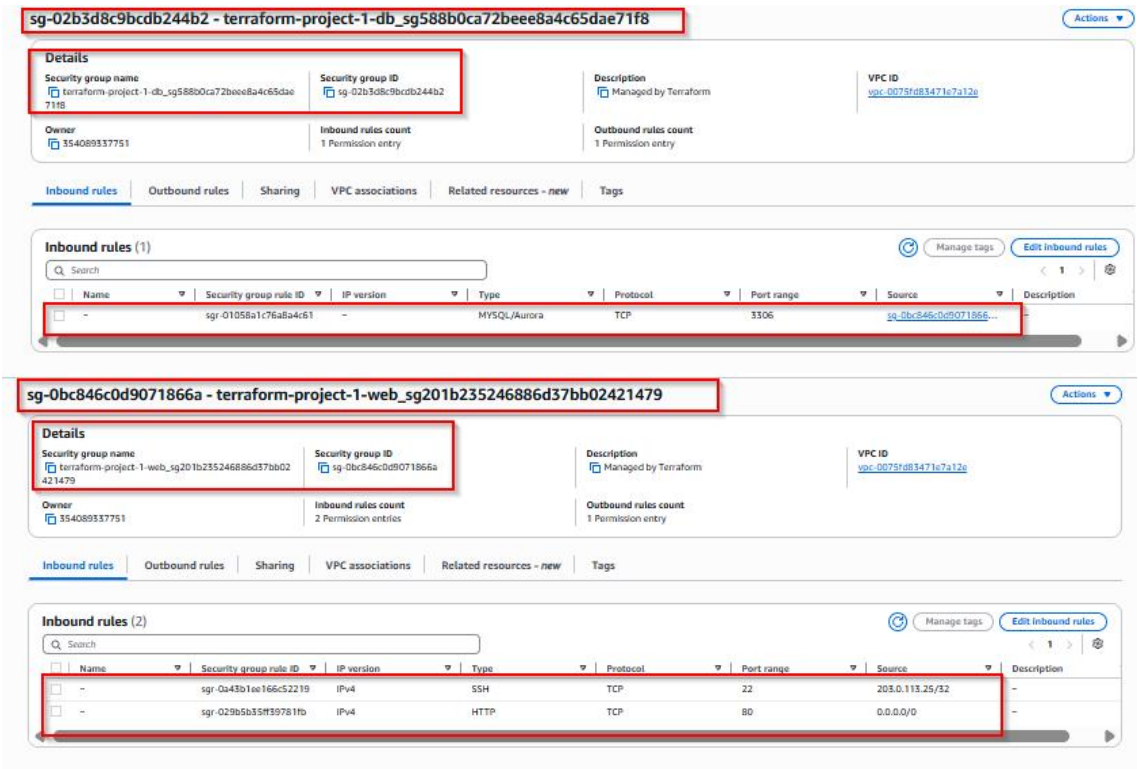


Figure 22: Web and Database Security Groups.

Step 19: EC2 Instances

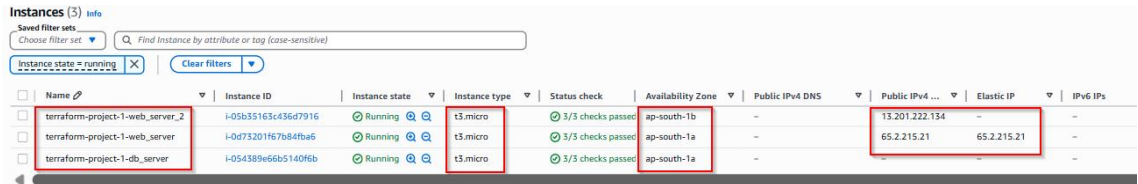


Figure 23: EC2 instances up & running

Step 20: AWS S3 Bucket

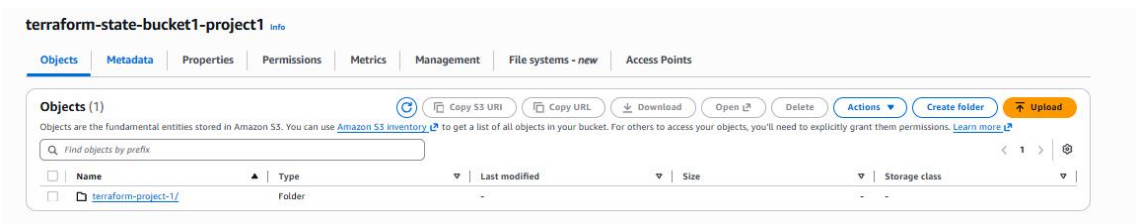


Figure 23: AWS S3 bucket created.

Step 21: key pair

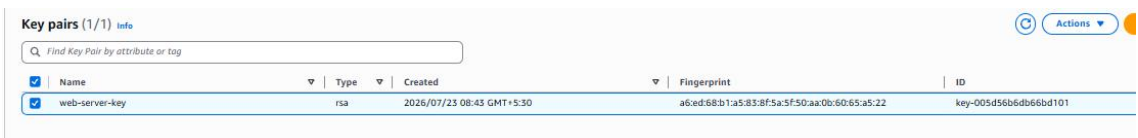


Figure 24: Key pair created using terraform

Step 22: web page outputs
First Web_server output:

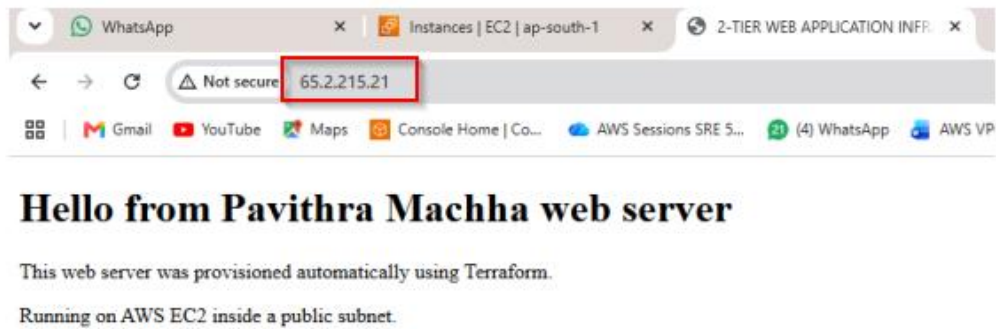


Figure 25:web page using web_server public ip

Second webserver output:

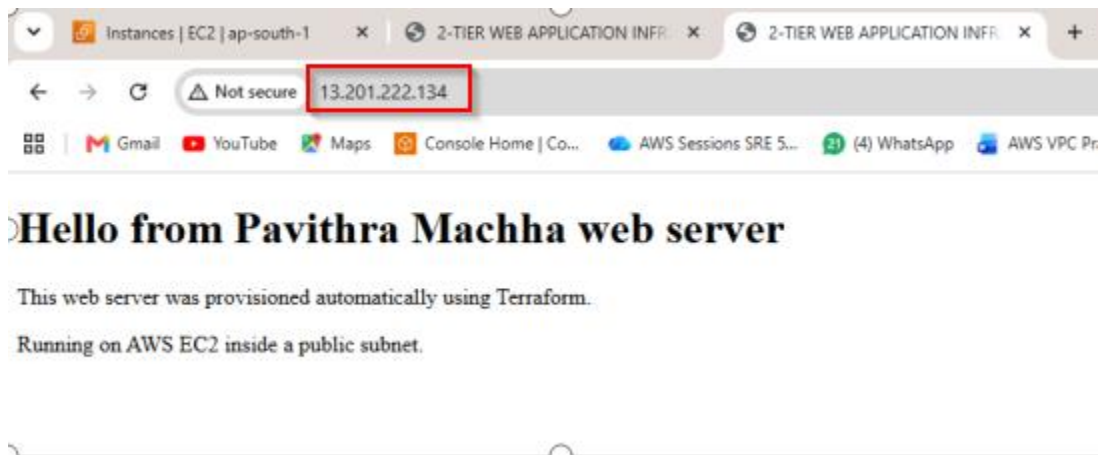


Figure 26:web page using web_server_2 public ip

Step 23:Terraform Output

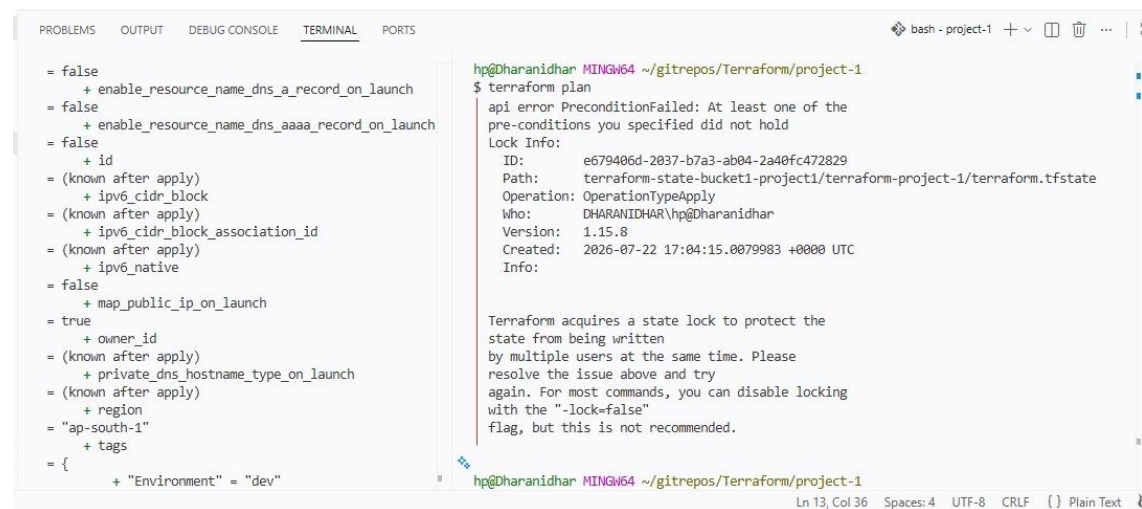
```
Outputs:

db_private_ip = "10.0.2.226"
vpc_id = "vpc-0075fd83471e7a12e"
web_public_ip = "65.2.215.21"
web_public_url = "http://65.2.215.21"

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
```

Figure 26: terraform outputs

Step 24:Terraform state locking



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
= false
+ enable_resource_name_dns_a_record_on_launch
= false
+ enable_resource_name_dns_aaaa_record_on_launch
= false
+ id
= (known after apply)
+ ipv6_cidr_block
= (known after apply)
+ ipv6_cidr_block_association_id
= (known after apply)
+ ipv6_native
= false
+ map_public_ip_on_launch
= true
+ owner_id
= (known after apply)
+ private_dns_hostname_type_on_launch
= (known after apply)
+ region
= "ap-south-1"
+ tags
= {
+ "Environment" = "dev"

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
$ terraform plan
api error PreconditionFailed: At least one of the
pre-conditions you specified did not hold
Lock Info:
ID: e679406d-2037-b7a3-ab04-2a40fc472829
Path: terraform-state-bucket1-project1/terraform-project-1/terraform.tfstate
Operation: OperationTypeApply
Who: DHARANIDHAR\hp@Dharanidhar
Version: 1.15.8
Created: 2026-07-22 17:04:15.0079983 +0000 UTC
Info:

Terraform acquires a state lock to protect the
state from being written
by multiple users at the same time. Please
resolve the issue above and try
again. For most commands, you can disable locking
with the "-lock=false"
flag, but this is not recommended.

hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
Ln 13, Col 36 Spaces: 4 UTF-8 CRLF {} Plain Text
```

Figure 27:Terraform state locked.

Step 25:Terraform destroy



```
hp@Dharanidhar MINGW64 ~/gitrepos/Terraform/project-1
$ terraform destroy
aws_instance.web_server_2: Still destroying... [id=i-05b35163c436d7916, 01m20s elapsed]
aws_instance.web_server_2: Destruction complete after 1m24s
aws_subnet.public_subnet_2: Destroying... [id=subnet-03b29825d7d121330]
aws_security_group.web_sg: Destroying... [id=sg-0bc846c0d9071866a]
aws_subnet.public_subnet_2: Destruction complete after 1s
aws_security_group.web_sg: Destruction complete after 1s
aws_vpc.main: Destroying... [id=vpc-0075fd83471e7a12e]
aws_vpc.main: Destruction complete after 1s
Releasing state lock. This may take a few moments...

Destroy complete! Resources: 16 destroyed.
```

Figure 28:Successful terraform destroy

Project Outcome

The project successfully automated AWS infrastructure deployment using Terraform. The webserver was accessible through a public ip, while the database server remained securely deployed in a private subnet. All networking, security, and compute resources were created and managed through infrastructure as Code(IaC).

Conclusion

The Aws infrastructure for project-1 was successfully provisioned using Terraform and verified through both the Terraform CLI and the AWS Management Console. All required networking, Security, and computer resources were created according to the project requirements. The webserver was accessible via its public IP, while the database server remained securely isolated in the private subnet. Finally, all resources were removed successfully using Terraform destroy.